Polisci 270 Final Cheat Sheet

Fayzaan Virk

2026-05-01

Contents

Data Wrangling . . . 2
- group_by() and summarize() . . . 2
- filter() and select() . . . 2
- mutate() and rename() . . . 3
- arrange() . . . 3

Average Treatment Effect . . . 4

Ggplot . . . 5
- Scatterplot . . . 5
- Bar plots . . . 6
- Histogram . . . 7
- Box and Whisker Plot . . . 8
- Colors . . . 9

Experimental Types . . . 10
- Research Designs . . . 10

Proportion Tables . . . 11

Z-Score . . . 11

Linear Models . . . 12
- Categorical and binary variables . . . 13

Bootstrapping . . . 13
- Dplyr bootstrapping . . . 13
- Infer bootstrapping . . . 14

Confidence Intervals . . . 16

Hypothesis Testing . . . 16

Normal Distributions . . . 16

Data Wrangling

Data wrangling is the process of taking a complicated data set and making it easier to work with. This can mean filtering or selecting only a certain number or rows or columns, adding new variables, or pivoting the table. In R, we use the group_by(), summarize(), pivot_wider(), mutate(), filter(), select(), pivot_longer(), and the three join() commands to wrangle our data.

I will use the gapminder data set to demonstrate some of these commands

group_by() and summarize()

First, I can use the group_by() and summarize() commands to find the mean GDP per capita for each continent.

```
gapminder |>
  group_by(continent) |>
  summarize(mean_gdppc = mean(gdpPercap))
```

```
## # A tibble: 5 x 2
##   continent mean_gdppc
##   <fct>          <dbl>
## 1 Africa         2194.
## 2 Americas       7136.
## 3 Asia           7902.
## 4 Europe        14469.
## 5 Oceania       18622.
```

filter() and select()

I can use the filter() command to only see certain rows of the data set.

```
gapminder |>
  filter(year == 2007)
```

```
## # A tibble: 142 x 6
##    country     continent  year lifeExp       pop gdpPercap
##    <fct>       <fct>     <int>   <dbl>     <int>     <dbl>
##  1 Afghanistan Asia       2007    43.8  31889923      975.
##  2 Albania     Europe     2007    76.4   3600523     5937.
##  3 Algeria     Africa     2007    72.3  33333216     6223.
##  4 Angola      Africa     2007    42.7  12420476     4797.
##  5 Argentina   Americas   2007    75.3  40301927    12779.
##  6 Australia   Oceania    2007    81.2  20434176    34435.
##  7 Austria     Europe     2007    79.8   8199783    36126.
##  8 Bahrain     Asia       2007    75.6    708573    29796.
##  9 Bangladesh  Asia       2007    64.1 150448339     1391.
## 10 Belgium     Europe     2007    79.4  10392226    33693.
## # i 132 more rows
```

The select() command works the same way as filter() but for choosing certain columns

```r
gapminder |>
  filter(year == 2007) |>
  select(country, gdpPercap)
```

```
## # A tibble: 142 x 2
##    country     gdpPercap
##    <fct>           <dbl>
##  1 Afghanistan      975.
##  2 Albania         5937.
##  3 Algeria         6223.
##  4 Angola          4797.
##  5 Argentina      12779.
##  6 Australia      34435.
##  7 Austria        36126.
##  8 Bahrain        29796.
##  9 Bangladesh      1391.
## 10 Belgium        33693.
## # i 132 more rows
```

mutate() and rename()

The mutate() command can create new columns using logicals or calculations and rename() can change existing columns' names.

```r
gapminder |>
  mutate(
    gdp = gdpPercap * pop,
    post_coldwar = if_else(year > 1991, 1, 0)
  ) |>
  rename("population" = pop)
```

```
## # A tibble: 1,704 x 8
##    country     continent  year lifeExp population gdpPercap     gdp post_coldwar
##    <fct>       <fct>     <int>   <dbl>      <int>     <dbl>   <dbl>        <dbl>
##  1 Afghanistan Asia       1952    28.8    8425333      779. 6.57e 9            0
##  2 Afghanistan Asia       1957    30.3    9240934      821. 7.59e 9            0
##  3 Afghanistan Asia       1962    32.0   10267083      853. 8.76e 9            0
##  4 Afghanistan Asia       1967    34.0   11537966      836. 9.65e 9            0
##  5 Afghanistan Asia       1972    36.1   13079460      740. 9.68e 9            0
##  6 Afghanistan Asia       1977    38.4   14880372      786. 1.17e10            0
##  7 Afghanistan Asia       1982    39.9   12881816      978. 1.26e10            0
##  8 Afghanistan Asia       1987    40.8   13867957      852. 1.18e10            0
##  9 Afghanistan Asia       1992    41.7   16317921      649. 1.06e10            1
## 10 Afghanistan Asia       1997    41.8   22227415      635. 1.41e10            1
## # i 1,694 more rows
```

arrange()

The arrange() function will reorder the entries in a column

```r
gapminder |>
  filter(year == 2007) |>
  arrange(desc(gdpPercap))
```

```
## # A tibble: 142 x 6
##    country          continent  year lifeExp       pop gdpPercap
##    <fct>            <fct>     <int>   <dbl>     <int>     <dbl>
##  1 Norway           Europe     2007    80.2   4627926    49357.
##  2 Kuwait           Asia       2007    77.6   2505559    47307.
##  3 Singapore        Asia       2007    80.0   4553009    47143.
##  4 United States    Americas   2007    78.2 301139947    42952.
##  5 Ireland          Europe     2007    78.9   4109086    40676.
##  6 Hong Kong, China Asia       2007    82.2   6980412    39725.
##  7 Switzerland      Europe     2007    81.7   7554661    37506.
##  8 Netherlands      Europe     2007    79.8  16570613    36798.
##  9 Canada           Americas   2007    80.7  33390141    36319.
## 10 Iceland          Europe     2007    81.8    301931    36181.
## # i 132 more rows
```

Add in a section for join() and pivot_longer()

See Code from Lecture 4, 5, 9, 12, Problem Set 2, 3, 4, 5, 6, 8

Average Treatment Effect

The average treatment effect is a way of calculation how much a treatment affects the outcome variable. The formula is the expected outcome variable if all subjects were treated minus the expected outcome if all subjects were not treated. To approximate this, we use the mean of the treatment group for the outcome variable minus the mean of the control group.

I'll use the trains experiments (Spanish-speakers in train stations in Boston).

```r
trains |>
  group_by(treatment) |>
  summarize(post_mean = mean(numberim.post)) |>
  pivot_wider(
    names_from = treatment,
    values_from = post_mean
  ) |>
  mutate(
    ate = `1` - `0`
  )
```

```
## # A tibble: 1 x 3
##     `0`   `1`   ate
##   <dbl> <dbl> <dbl>
## 1  2.73  3.12 0.383
```

Additionally, you can use select() and pull() to get just the numeric value

```r
trains |>
  group_by(treatment) |>
  summarize(post_mean = mean(numberim.post)) |>
  pivot_wider(
    names_from = treatment,
    values_from = post_mean
  ) |>
  mutate(
    ate = `1` - `0`
  ) |>
  select(ate) |>
  pull()
```

```
## [1] 0.3832721
```

See Code from Lecture 7, Problem Set 3, 7

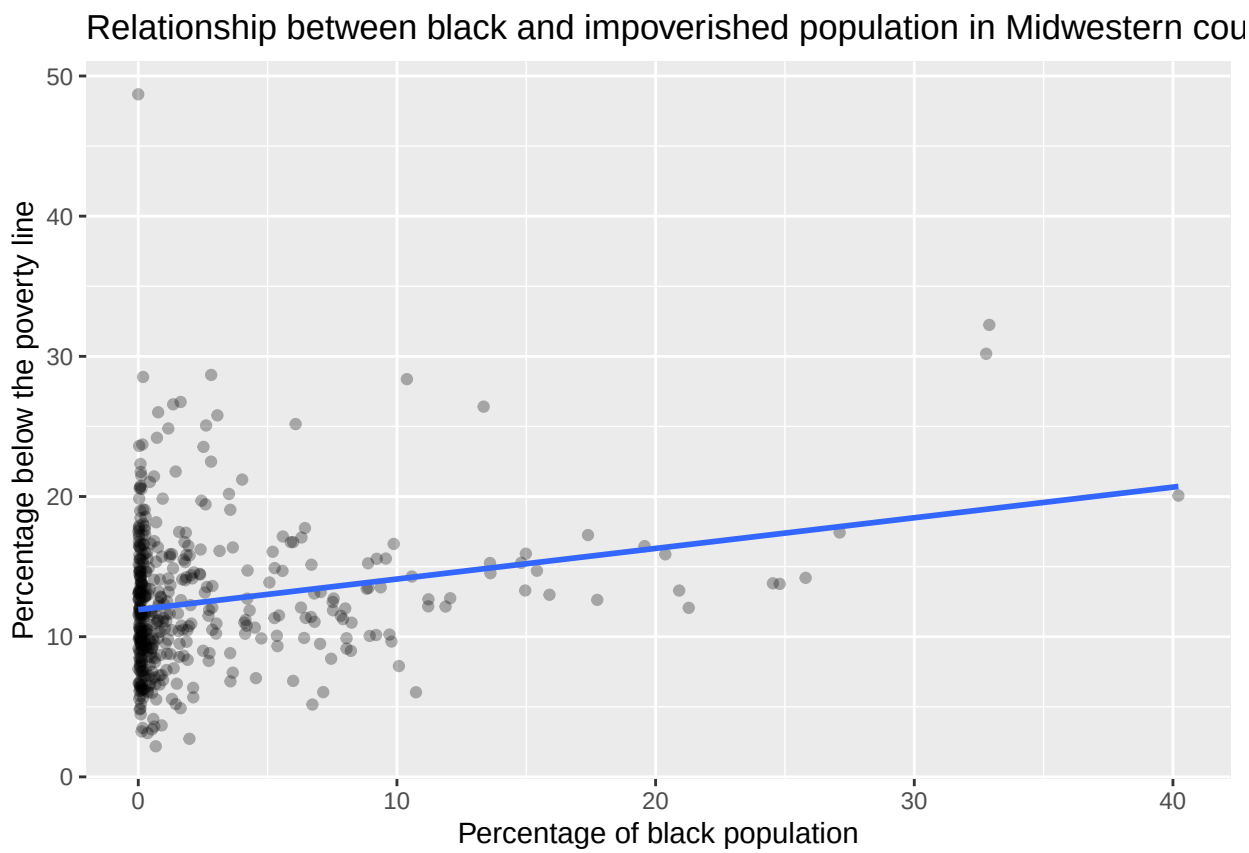
Ggplot

ggplot2 is a package that is part of tidyverse that is used to create many types of graphs and visual representations of data. In class, we learned scatterplots, box and whisker plots, barplots, and histograms. These are created using geom_point(), geom_box(), geom_bar() or geom_col(), and geom_histogram(). For a scatterplot, a line of best fit can be added using geom_smooth().

Scatterplot

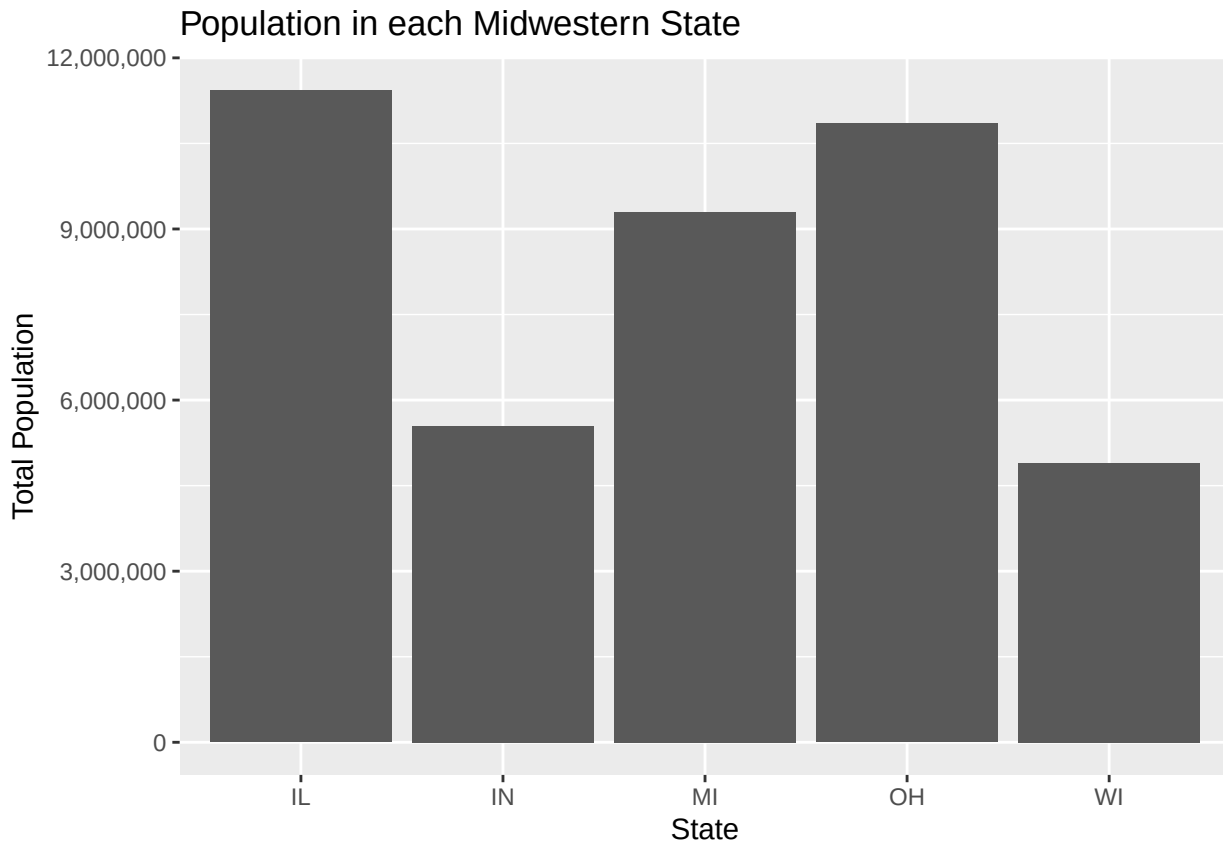
```r
midwest |>
  ggplot(aes(x = percblack, y = percbelowpoverty)) +
  geom_point(alpha = 0.3) +
  geom_smooth(method = "lm", se = F) +
  labs(
    x = "Percentage of black population",
    y = "Percentage below the poverty line",
    title = "Relationship between black and impoverished population in Midwestern counties"
  )
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



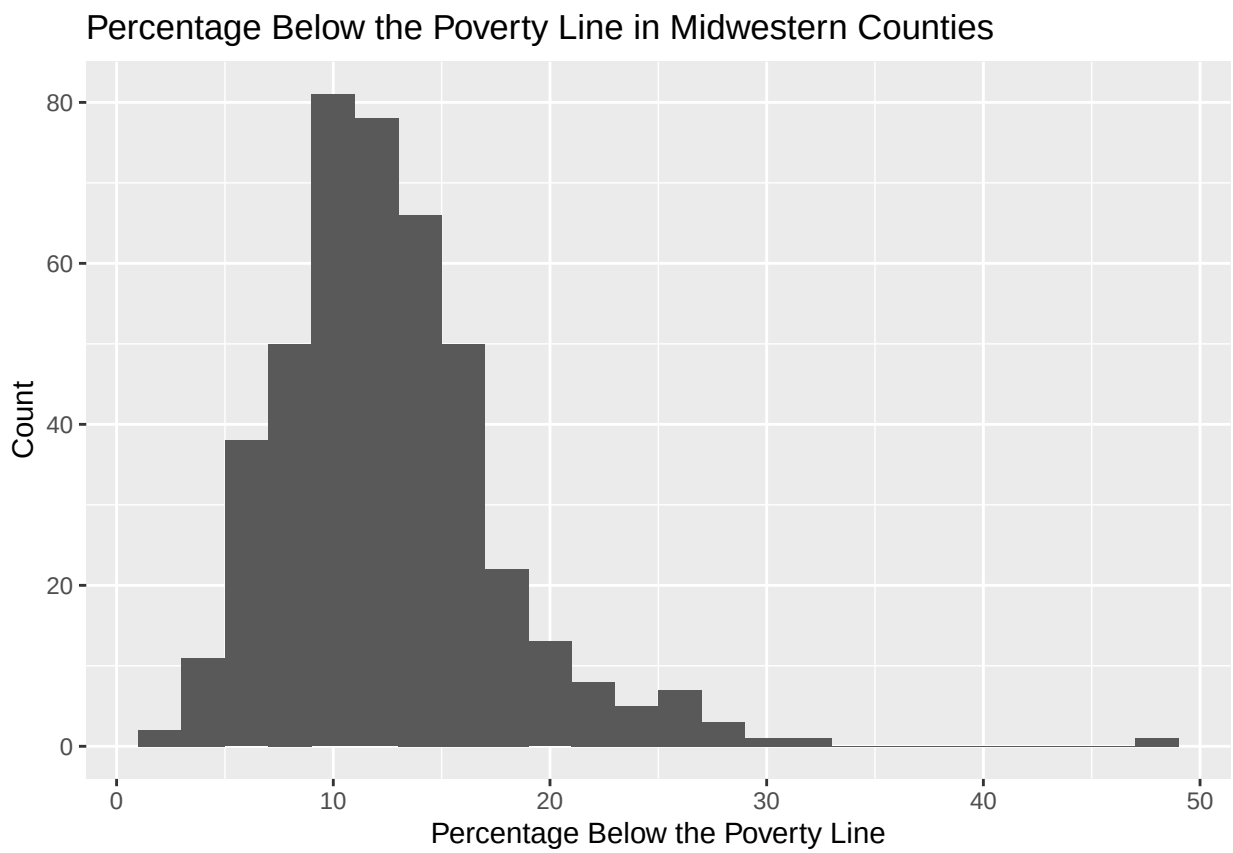
Bar plots

```
midwest |>
  group_by(state) |>
  summarize(population = sum(poptotal)) |>
  ggplot(aes(x = state, y = population)) +
  geom_col() +
  scale_y_continuous(labels = scales::comma) +
  labs(
    x = "State",
    y = "Total Population",
    title = "Population in each Midwestern State"
  )
```



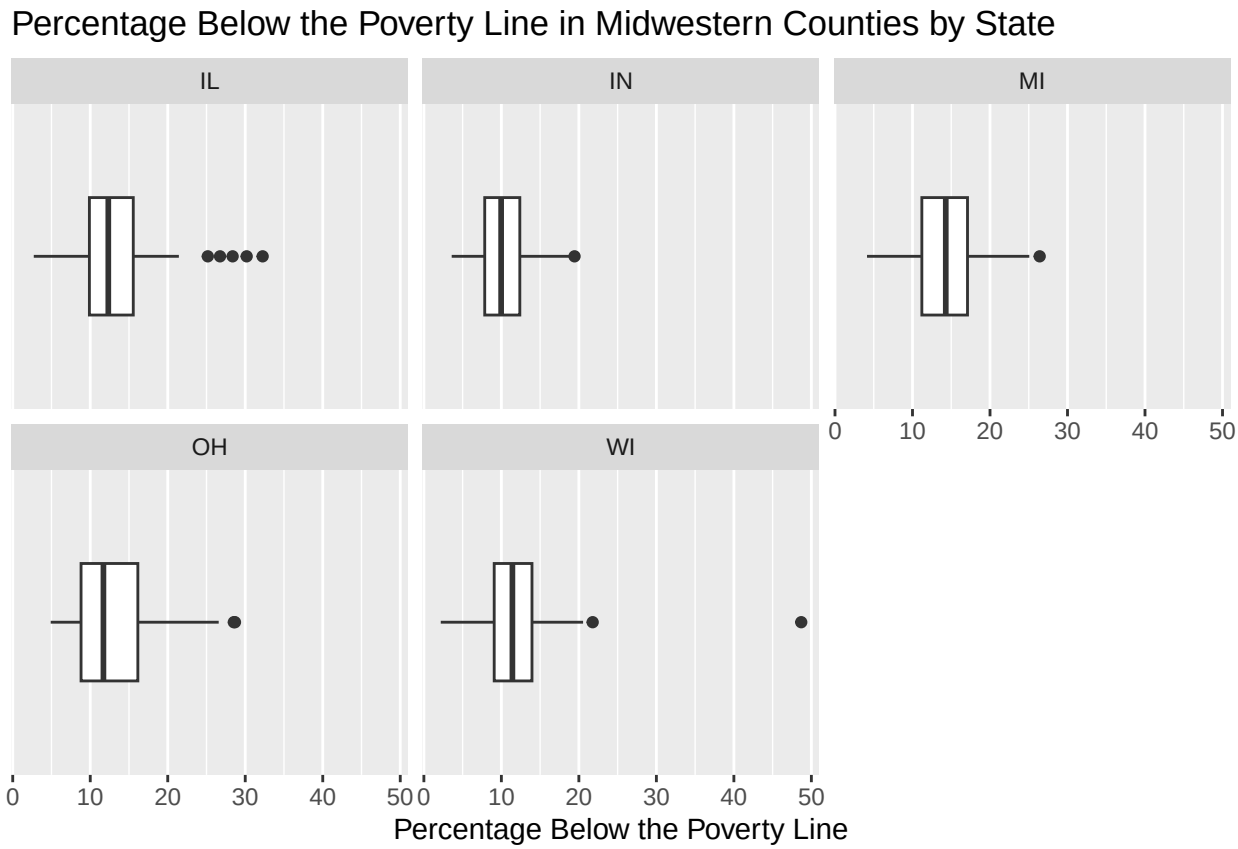
Histogram

```
midwest |>
  ggplot(aes(x = percbelowpoverty)) +
  geom_histogram(binwidth = 2) +
  labs(
    x = "Percentage Below the Poverty Line",
    y = "Count",
    title = "Percentage Below the Poverty Line in Midwestern Counties"
  )
```



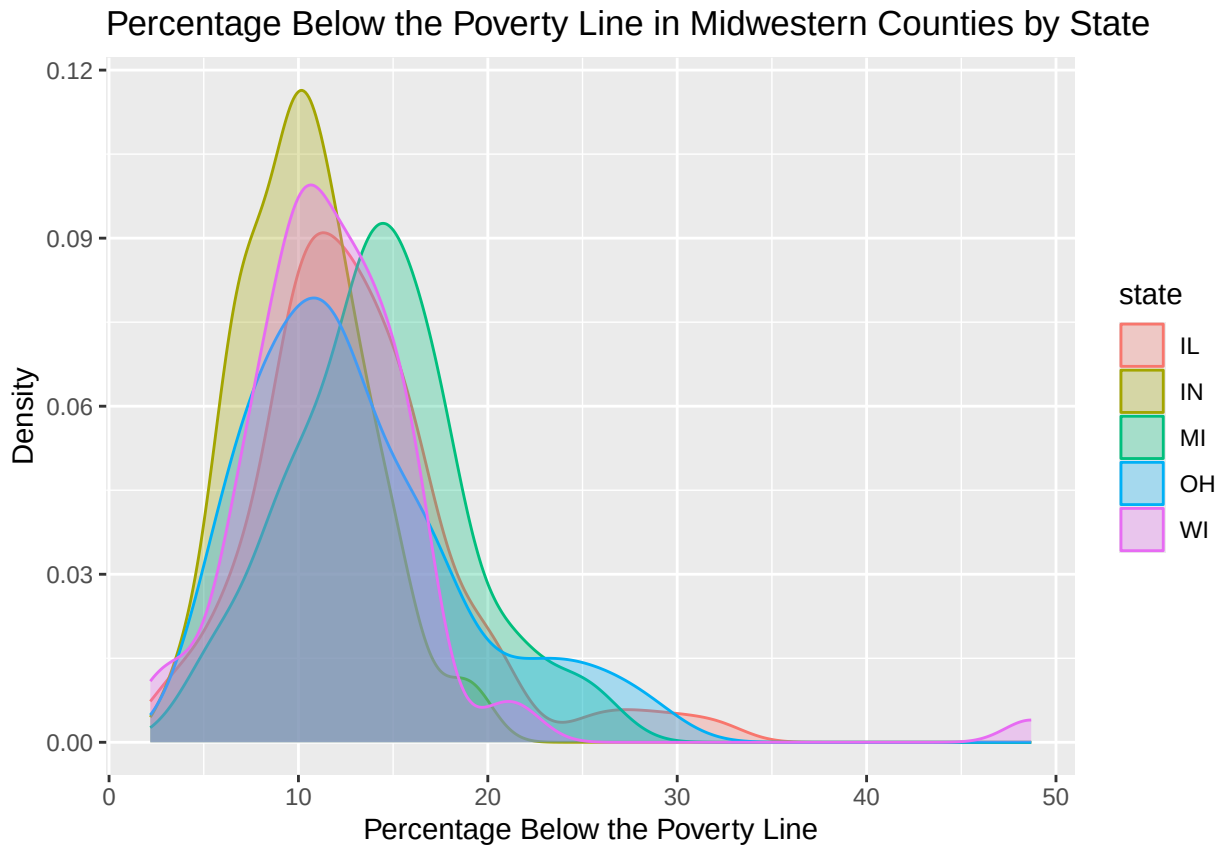
Box and Whisker Plot

```
midwest |>
  ggplot(aes(x = percbelowpoverty)) +
  geom_boxplot() +
  facet_wrap(~ state) +
  scale_y_discrete() +
  labs(
    x = "Percentage Below the Poverty Line",
    title = "Percentage Below the Poverty Line in Midwestern Counties by State"
  )
```



Colors

```
midwest |>
  ggplot(
    aes(
      x = percbelowpoverty,
      fill = state,
      color = state
    )) +
  geom_density(alpha = 0.3) +
  labs(
    x = "Percentage Below the Poverty Line",
    y = "Density",
    title = "Percentage Below the Poverty Line in Midwestern Counties by State"
  )
```



See Code from Lecture 3, Problem Set 1, 2, 3, 4, 5, 6, 7

Experimental Types

An observational study is where we simply observe how two variables interact in real life. We do not assign a treatment. This is very common in real life, since we often can't assign treatments due to ethical or logistical reasons. Because we cannot randomly assign treatments, it is much harder to establish causality from an observational study because of a greater chance of confounding variables. On the other hand, Randomized Controlled Trials (RCTs) do have randomly assigned treatment groups. RCTs tend to have a higher internal validity, meaning it is easier to conclude a causal relationship from them. Observational studies tend to have a higher external validity, which means the conclusion can be more easily generalized beyond this study.

Research Designs

There are three types of research designs. A cross-sectional design compares the control and treatment groups after the treatment has been administered. This assumes the treatment and control groups are on average identical. A before-and-after design compares the treatment group from before and after the treatment was administered. This assumes none of the confounders will vary with time. A difference-in-differences design used the before-and-after of the control group to infer what would've happened to the treatment group if no treatment was administered. This assumes that the trends that affect one group equally affect the other group. All three rely on assumptions that can't always be verified, and are thus more susceptible to confounding variables than a RCT. There are some ways to try and control for confounding variables using statistical procedures, one of which is subclassification. Subclassification is where treated and control groups are compared within the levels of a confounder.

See Code from Lecture 8, Problem Set 3

Proportion Tables

```
cces_2020 |>
  group_by(pres_vote) |>
  summarize(n = n()) |>
  mutate(proportion = n / sum(n))
```

```
## # A tibble: 7 x 3
##   pres_vote                        n proportion
##   <fct>                        <int>      <dbl>
## 1 Joe Biden (Democrat)         26188   0.508
## 2 Donald J. Trump (Republican) 17702   0.343
## 3 Other                         1458   0.0283
## 4 I did not vote in this race    100   0.00194
## 5 I did not vote                  13   0.000252
## 6 Not sure                       190   0.00369
## 7 <NA>                          5900   0.114
```

For multiple grouping variables, see below

```
cces_2020 |>
  filter(pres_vote %in% c("Joe Biden (Democrat)", "Donald J. Trump (Republican)")) |>
  mutate(pres_vote = if_else(pres_vote == "Joe Biden (Democrat)", "Biden", "Trump")) |>
  group_by(pid3, pres_vote) |>
  summarize(n = n()) |>
  mutate(prop = n/sum(n)) |>
  select(!n) |>
  pivot_wider(
    names_from = pid3,
    values_from = prop
  ) |>
  knitr::kable()
```

```
## `summarise()` has grouped output by 'pid3'. You can override using the
## `.groups` argument.
```

pres_vote	Democrat	Republican	Independent	Other	Not sure
Biden	0.9681295	0.0712146	0.5714162	0.4870775	0.5993092
Trump	0.0318705	0.9287854	0.4285838	0.5129225	0.4006908

See Code from Lecture 9, 10

Z-Score

The z-score represent how many standard deviations away from the mean a data point is. The formula is (x - mu)/sigma, where mu is variable's mean and sigma is the variable's standard deviation. Below I will find the z-score for Cook County, IL for the proportion of people 5 and older who got at least one dose of the COVID vaccine.

```r
dose5_stats <- covid_votes |>
  summarize(
    mean_dose5 = mean(one_dose_5plus_pct, na.rm=T),
    sd_dose5 = sd(one_dose_5plus_pct, na.rm=T)
  )

cook_dose5 <- covid_votes |>
  filter(county == "Cook County" & state == "IL") |>
  select(one_dose_5plus_pct) |>
  pull()

cook_z = (cook_dose5 - dose5_stats$mean_dose5) / dose5_stats$sd_dose5
cook_z
```

```
## [1] 1.695176
```

See Code from Lecture 11

Linear Models

A linear model is a great way to see if one variable has a positive, negative, or no effect on another variable. A linear model can be created using the lm() command. The parameters are lm(dependent variable(s) ~ independent, data set).

```r
lobf <- lm(percbelowpoverty ~ percblack, midwest)
coef(lobf)
```

```
## (Intercept)   percblack
##  11.9255333   0.2185736
```

For a county with a black percentage of zero, the percentage below the poverty line is predicted to be 11.93% For every percentage point increase in the black population, the percentage below the poverty line is predicted to increase by 0.22 percentage points.

```r
lobf_2 <- lm(percbelowpoverty ~ percblack + percasian + popdensity + poptotal, midwest)
coef(lobf_2)
```

```
##   (Intercept)     percblack     percasian    popdensity      poptotal
##  1.220449e+01  4.231014e-01 -3.700647e-01 -2.782850e-04  2.247291e-06
```

For a county with a black percentage of 0, and Asian percentage of 0, a population density of 0, and a total population of zero, the percentage below the poverty line is expected to be 12.2%. For every percentage point increase in the black population, the percentage below the poverty line is predicted to increase by 0.42 percentage points. For every percentage point increase in the Asian population, the percentage below the poverty line is predicted to decrease by 0.37 percentage points. For every increase by one in population density, the percentage below the poverty line is predicted to decrease by 0.0002 percentage points. For every increase by 1000 in the total population, the percentage below the poverty line is predicted to increase by 0.002 percentage points. Making a linear model with multiple variables can help us isolate confounding effect by comparing the different effects multiple response variables have on the outcome. In the example above, we see one potential confounder could be total population, and it becomes more difficult to draw a causal relationship between black population and percentage beneath the poverty line.

Categorical and binary variables

A linear model can be used to calculate the ATE if and only if the independent variable is a binary variable. In this case, the ATE would be the slope of the regression line. A linear model can also be used for categorical variables.

```
lobf_3 <- lm(nondiscrim_post ~ racename, trans)
coef(lobf_3)
```

```
##             (Intercept)           racenameAsian       racenameCaucasian
##              0.63492063              0.19841270              0.06244779
##        racenameHispanic racenameNative American
##              0.02936508             -0.63492063
```

This linear model means that the proportion of African Americans that didn't discriminate post-treatment was 63.5%. Every coefficient is how other races compare to that score. For example, the proportion of Asians that didn't discriminate post-treatment is 19.8 percentage points highet and the percentage of Native Americans that didn't discriminate post-treatment was 6.3 percentage points lower.

See Code from Lecture 14, 15, 16, 25, Problem Set 5

Bootstrapping

Dplyr bootstrapping

Bootstrapping is the process of creating a sampling distribution from a single sample. It is incredibly useful because in many cases, we don't have access to the full population data or multiple samples. It creates multiple samples of the same size as the one we have be repeatedly drawing with replacement from our sample. This creates a sampling distribution of the same shape (spread) as the population sampling distribution would be, but centered at the sample statistic. We can then use the bootstrap for calculating confidence intervals or hypothesis tests because it's shape is the same.

I will show both the dplyr and infer workflows for making a bootstrap to calculate the proportion of all Americans who want the government to do more about the environment using one sample from the American national election survey.

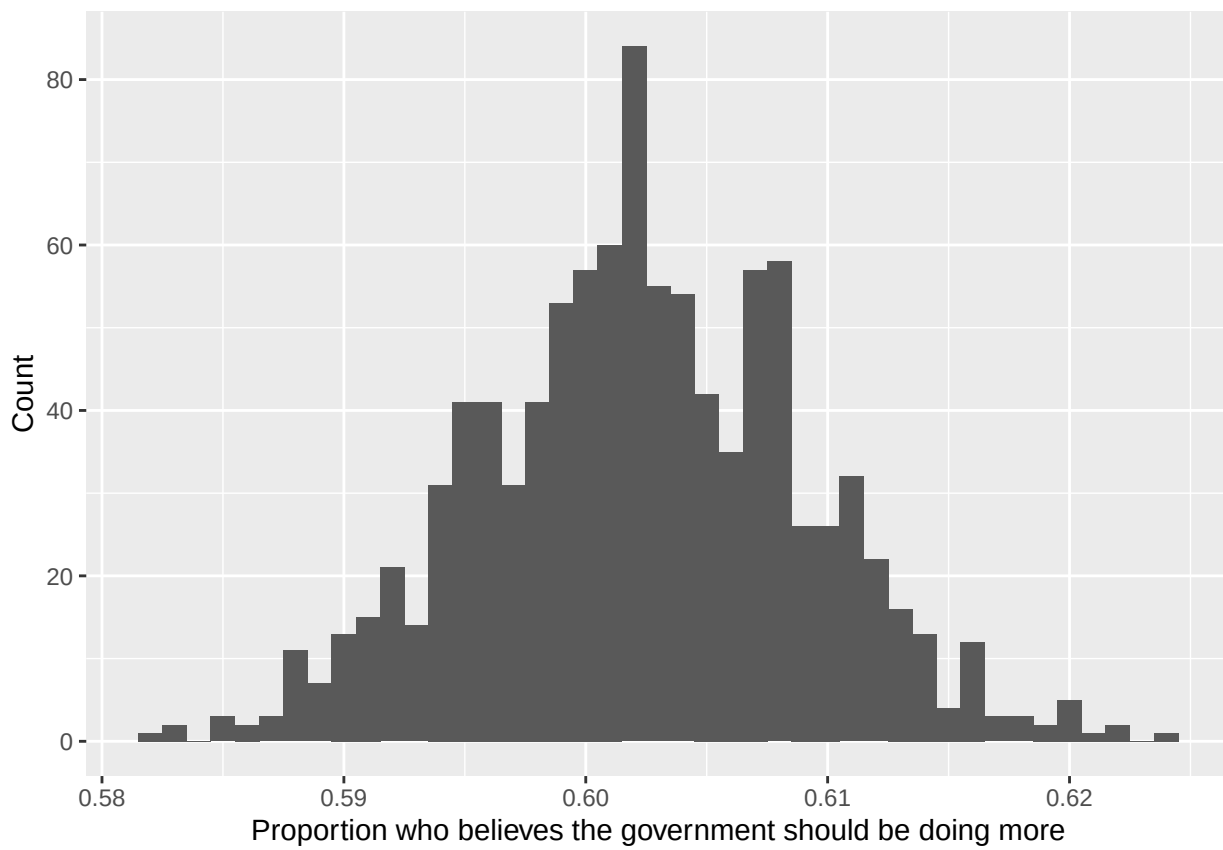
```
set.seed(53132)
dplyr_boots <- anes |>
  rep_slice_sample(reps = 1000, prop = 1, replace = T) |>
  group_by(replicate) |>
  summarize(mean_envir = mean(envir_doing_more))

dplyr_boots
```

```
## # A tibble: 1,000 x 2
##    replicate mean_envir
##        <int>      <dbl>
##  1         1      0.602
##  2         2      0.595
##  3         3      0.601
##  4         4      0.609
##  5         5      0.595
##  6         6      0.607
```

```
## 7          7      0.595
## 8          8      0.590
## 9          9      0.595
## 10        10      0.610
## # i 990 more rows
```

When being visualized, this bootstrap distribution should have an approximately normal distribution.

```
dplyr_boots |>
  ggplot(aes(x = mean_envir)) +
  geom_histogram(binwidth = 0.001) +
  labs(
    x = "Proportion who believes the government should be doing more",
    y = "Count"
  )
```



Infer bootstrapping

This can also be done using the infer package

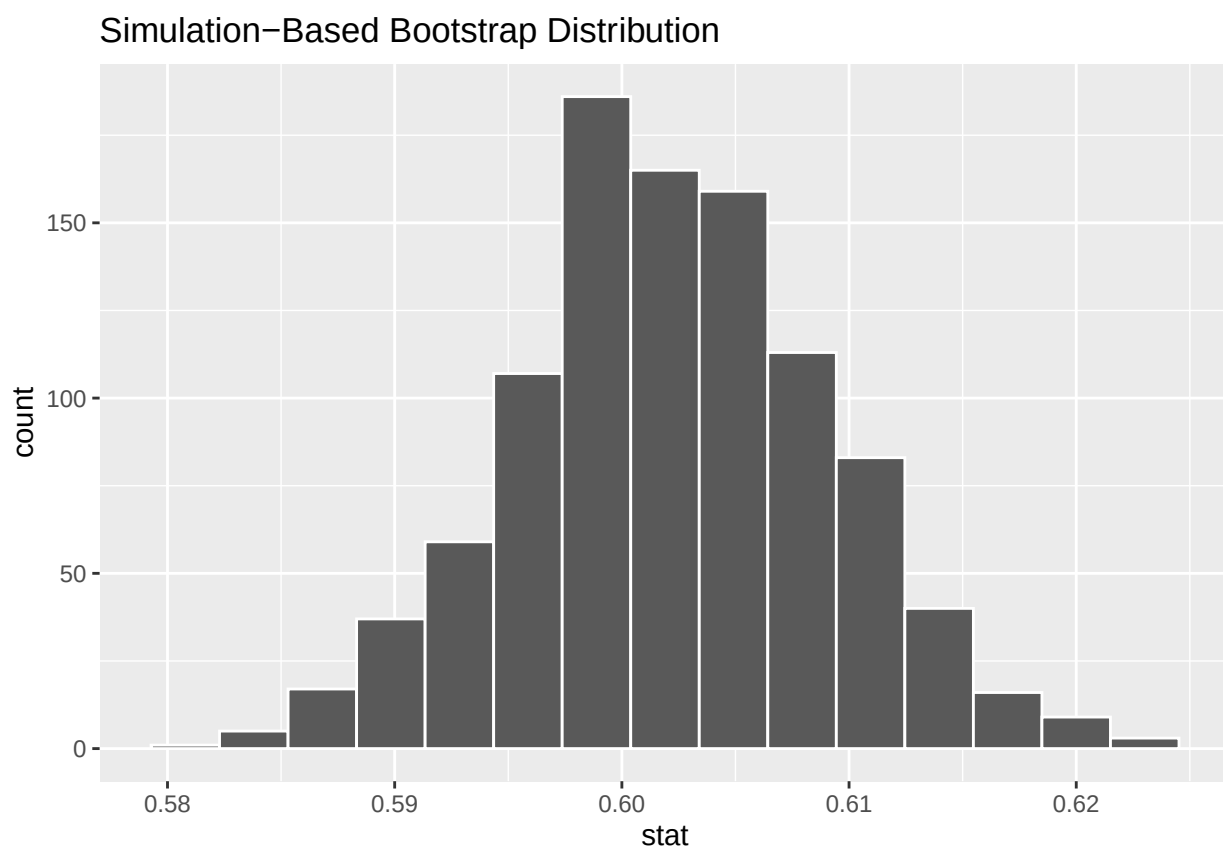
```
set.seed(60004)
infer_boots <- anes |>
  specify(response = envir_doing_more) |>
  generate(reps = 1000, type = "bootstrap") |>
```

```r
  calculate(stat = "mean")
infer_boots
```

```
## Response: envir_doing_more (numeric)
## # A tibble: 1,000 x 2
##    replicate  stat
##        <int> <dbl>
##  1         1 0.595
##  2         2 0.601
##  3         3 0.601
##  4         4 0.612
##  5         5 0.609
##  6         6 0.613
##  7         7 0.598
##  8         8 0.596
##  9         9 0.597
## 10        10 0.600
## # i 990 more rows
```

Infer can also be used to visualize the bootstrap distribution

```r
visualize(infer_boots)
```



See Code from Lecture 17, 19, Problem Set 6, 7, 8

Confidence Intervals

See Code from Lecture 20, Problem Set 7

Hypothesis Testing

See Code from Lecture 21, 22, 25, Problem Set 8

Normal Distributions

See Code from Lecture 23